

CampusHub

Piattaforma web per eventi e attività studentesche

1. Descrizione del progetto

CampusHub è una web application per la gestione di eventi e attività rivolte a studenti universitari.

La piattaforma consente di:

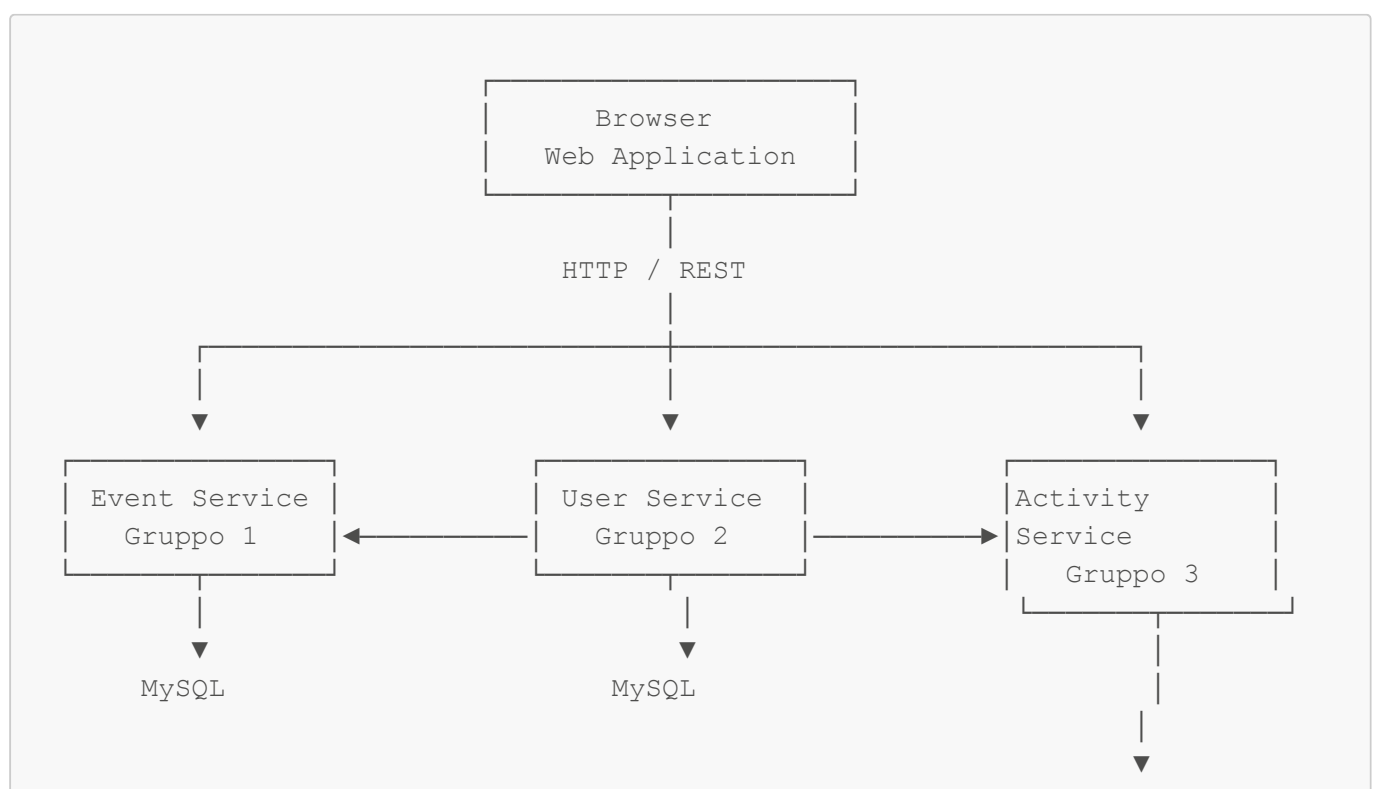
- visualizzare e ricercare eventi;
- filtrare gli eventi per categoria, data e località;
- creare e modificare eventi;
- gestire profili utente;
- associare interessi agli utenti;
- partecipare agli eventi;
- gestire le iscrizioni;
- ottenere suggerimenti personalizzati sugli eventi;
- visualizzare le attività e le partecipazioni dell'utente.

L'applicazione è composta da più servizi indipendenti che comunicano attraverso API REST.

Il progetto è suddiviso in **5 gruppi**, ciascuno responsabile di uno specifico componente applicativo.

2. Architettura generale

L'architettura prevista è composta dai seguenti componenti:



```
graph TD; subgraph Container; direction TB; G4[Recommendation Service Gruppo 4]; G5[Web Application / Frontend Gruppo 5]; end
```

Ogni servizio backend deve possedere e gestire autonomamente il proprio database.

Non è consentito l'accesso diretto al database di un altro servizio.

La comunicazione tra servizi deve avvenire esclusivamente tramite API REST.

3. Requisiti tecnologici comuni

Tutti i gruppi devono utilizzare:

Backend

- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- Maven

Database

- MySQL

API

- REST
- HTTP
- JSON
- OpenAPI / Swagger

Versionamento

- Git
- GitHub

Frontend

Sono ammessi:

- HTML
- CSS
- Bootstrap
- JavaScript
- Thymeleaf

oppure un framework frontend compatibile con l'architettura del progetto.

4. Gruppo 1 – Event Service

Obiettivo

Realizzare il servizio REST responsabile della gestione degli eventi presenti sulla piattaforma.

Entità Event

L'entità deve contenere almeno:

```
Event
├── id
├── title
├── description
├── category
├── location
├── dateTime
├── maxParticipants
├── organizerId
└── status
```

Le categorie previste sono:

```
PARTY
SPORT
MUSIC
STUDY
WORKSHOP
CULTURE
OTHER
```

Lo stato dell'evento può prevedere, ad esempio:

```
DRAFT
PUBLISHED
```

```
CANCELLED  
COMPLETED
```

API REST obbligatorie

```
GET      /api/events  
GET      /api/events/{id}  
POST     /api/events  
PUT      /api/events/{id}  
DELETE   /api/events/{id}
```

Ricerca e filtri

Devono essere disponibili filtri almeno per:

```
GET /api/events?category=MUSIC  
  
GET /api/events?location=Torino  
  
GET /api/events?from=2026-10-01  
  
GET /api/events?to=2026-10-31
```

È possibile prevedere la combinazione di più filtri.

Requisiti

Il servizio deve:

- utilizzare Spring Data JPA;
- utilizzare MySQL;
- validare i dati ricevuti;
- utilizzare DTO per le API;
- restituire corretti HTTP status code;
- gestire gli errori;
- documentare le API con OpenAPI/Swagger;
- fornire dati di esempio per il test.

API utilizzate dagli altri gruppi

Il servizio deve consentire agli altri componenti di recuperare almeno:

```
GET /api/events/{id}  
GET /api/events
```

Il formato JSON delle risposte deve essere documentato.

5. Gruppo 2 – User & Profile Service

Obiettivo

Realizzare il servizio REST responsabile della gestione degli utenti, dei profili e degli interessi.

Entità User

```
User
├── id
├── username
├── email
├── name
├── surname
├── birthDate
├── university
└── bio
```

Entità Interest

```
Interest
├── id
└── name
```

Un utente può avere più interessi.

```
User 1 ————— * Interest
```

API User

```
GET    /api/users
GET    /api/users/{id}
POST   /api/users
PUT    /api/users/{id}
DELETE /api/users/{id}
```

API Interest

```
GET      /api/users/{id}/interests

POST     /api/users/{id}/interests

DELETE   /api/users/{id}/interests/{interestId}
```

Ricerca utenti

Devono essere disponibili almeno:

```
GET /api/users?university=...

GET /api/users?interest=music
```

Requisiti

Il servizio deve:

- utilizzare Spring Data JPA;
- utilizzare MySQL;
- utilizzare DTO;
- effettuare la validazione degli input;
- gestire correttamente gli errori;
- utilizzare HTTP status code appropriati;
- documentare le API con OpenAPI/Swagger;
- fornire dati di esempio.

API utilizzate dagli altri gruppi

Devono essere disponibili almeno:

```
GET /api/users/{id}

GET /api/users/{id}/interests
```

Il formato JSON delle risposte deve essere documentato.

6. Gruppo 3 – Activity / Booking Service

Obiettivo

Realizzare il servizio responsabile della gestione delle partecipazioni degli utenti agli eventi.

Il servizio deve utilizzare le API del:

- Gruppo 1 – Event Service;
- Gruppo 2 – User & Profile Service.

Non deve accedere direttamente ai database dei due servizi.

Entità Booking

```
Booking
├── id
├── userId
├── eventId
├── bookingDate
└── status
```

Lo stato può prevedere:

```
CONFIRMED
CANCELLED
```

API REST

```
GET    /api/bookings
GET    /api/bookings/{id}
POST   /api/bookings
PUT    /api/bookings/{id}
DELETE /api/bookings/{id}
```

Devono inoltre essere disponibili:

```
GET /api/users/{userId}/bookings
GET /api/events/{eventId}/bookings
```

Creazione di una partecipazione

La richiesta:

```
POST /api/bookings
```

può ricevere:

```
{  
  "userId": 15,  
  "eventId": 42  
}
```

Prima della creazione della partecipazione il servizio deve:

1. verificare che l'utente esista tramite User Service;
2. verificare che l'evento esista tramite Event Service;
3. verificare che l'evento sia disponibile;
4. verificare che l'utente non sia già iscritto;
5. verificare la disponibilità dei posti;
6. creare la partecipazione.

Comunicazione con gli altri servizi

Esempio:

```
Activity Service  
├── GET /api/users/15  
└── GET /api/events/42
```

Il servizio deve gestire anche il caso in cui uno dei servizi remoti non sia disponibile.

Requisiti

- Spring Boot
- Spring Data JPA
- MySQL
- REST
- JSON
- DTO
- gestione errori
- HTTP status code
- OpenAPI/Swagger

7. Gruppo 4 – Recommendation Service

Obiettivo

Realizzare un servizio che produca suggerimenti personalizzati sugli eventi disponibili.

Il servizio deve utilizzare le informazioni provenienti da:

- User Service;
- Event Service.

API principale

```
GET /api/recommendations/user/{userId}
```

La risposta deve contenere una lista di eventi consigliati.

Esempio:

```
[
  {
    "eventId": 42,
    "title": "Tech Night",
    "score": 0.92,
    "reason": "Matches your interest in technology"
  },
  {
    "eventId": 17,
    "title": "Campus Football",
    "score": 0.75,
    "reason": "Matches your interest in sport"
  }
]
```

Algoritmo minimo

La raccomandazione deve essere determinata sulla base di almeno:

- interessi dell'utente;
- categoria dell'evento;
- disponibilità dell'evento.

È possibile introdurre un sistema di scoring.

Esempio:

interesse coincidente	+50 punti
categoria preferita	+30 punti
evento recente	+10 punti
evento popolare	+10 punti

Il risultato deve essere ordinato per punteggio.

Estensioni opzionali

Il servizio può successivamente utilizzare ulteriori informazioni:

- eventi frequentati;
- categorie preferite;
- frequenza di partecipazione;
- popolarità degli eventi;
- località;
- distanza;
- storico delle prenotazioni.

Requisiti

- Spring Boot
 - REST
 - JSON
 - MySQL
 - Spring Data JPA
 - DTO
 - comunicazione REST con altri servizi
 - OpenAPI/Swagger
-

8. Gruppo 5 – Web Application

Obiettivo

Realizzare l'interfaccia web di CampusHub e integrare i servizi REST sviluppati dagli altri gruppi.

La Web Application deve comunicare con i servizi esclusivamente attraverso le relative API.

Non è consentito l'accesso diretto ai database dei servizi.

Funzionalità richieste

Home page

Deve mostrare:

- elenco degli eventi;
- categorie;
- eventi in evidenza;
- eventi consigliati.

Ricerca eventi

Deve consentire:

- ricerca per titolo;
- filtro per categoria;

- filtro per località;
- filtro per data.

Dettaglio evento

Deve visualizzare:

- titolo;
- descrizione;
- categoria;
- luogo;
- data;
- organizzatore;
- numero massimo di partecipanti;
- disponibilità;
- stato dell'evento.

Deve essere disponibile un comando:

```
PARTECIPA
```

per effettuare una prenotazione.

Profilo utente

Deve visualizzare:

- dati personali;
- università;
- biografia;
- interessi;
- eventi a cui l'utente partecipa.

Eventi consigliati

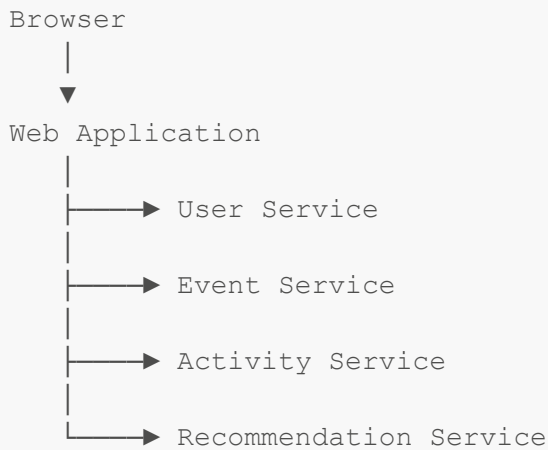
Deve essere presente una sezione:

```
Eventi consigliati per te
```

alimentata dal Recommendation Service.

Integrazione

Il flusso principale deve essere:



9. Contratti API

I Gruppi 1 e 2 devono definire e documentare i contratti delle proprie API.

Ogni API deve specificare:

- URL;
- metodo HTTP;
- parametri;
- request body;
- response body;
- HTTP status code;
- error response;
- esempi JSON.

Esempio:

```
GET /api/events/42
```

Response:

```
{
  "id": 42,
  "title": "Tech Night",
  "category": "TECHNOLOGY",
  "location": "Torino",
  "dateTime": "2026-10-12T20:30:00",
  "maxParticipants": 100,
  "organizerId": 15,
  "status": "PUBLISHED"
}
```

La documentazione deve essere disponibile tramite Swagger/OpenAPI.

10. Regole di integrazione

Ogni servizio deve essere autonomo.

Database

Ogni servizio utilizza il proprio database:

```
event_service_db
user_service_db
activity_service_db
recommendation_service_db
```

Non è consentito:

```
Activity Service
  |
  ▼
user_service_db
```

L'accesso ai dati deve avvenire tramite API:

```
Activity Service
  |
  | HTTP
  ▼
User Service
  |
  ▼
user_service_db
```

11. Gestione degli errori

Le API devono utilizzare correttamente gli HTTP status code.

Esempi:

```
200 OK
201 CREATED
```

```
204 NO CONTENT
400 BAD REQUEST
404 NOT FOUND
409 CONFLICT
500 INTERNAL SERVER ERROR
```

Le risposte di errore devono avere un formato coerente.

Esempio:

```
{
  "status": 404,
  "error": "NOT_FOUND",
  "message": "Event 42 not found",
  "timestamp": "2026-10-12T15:30:00"
}
```

12. Struttura minima dei backend

Ogni progetto Spring Boot deve adottare una struttura organizzata almeno nei seguenti livelli:

```
controller/
service/
repository/
entity/
dto/
exception/
config/
```

Esempio:

```
EventController
  |
  ▼
EventService
  |
  ▼
EventRepository
  |
  ▼
Event
```

Le API non devono contenere direttamente la logica di accesso al database.

13. Repository Git

Ogni gruppo deve utilizzare un repository Git dedicato.

Struttura indicativa:

```
campushub-event-service  
campushub-user-service  
campushub-activity-service  
campushub-recommendation-service  
campushub-web
```

Ogni repository deve contenere almeno:

```
README.md  
pom.xml  
src/
```

Il README deve descrivere:

- finalità del servizio;
- tecnologie utilizzate;
- configurazione;
- database;
- API disponibili;
- modalità di avvio;
- eventuali dipendenze da altri servizi.

14. Configurazione

Le configurazioni relative a:

- database;
- porte;
- URL degli altri servizi;
- credenziali;

non devono essere hardcoded nel codice.

Devono essere gestite attraverso la configurazione Spring Boot.

Esempio:

```
server.port=8081

spring.datasource.url=jdbc:mysql://localhost:3306/event_service_db

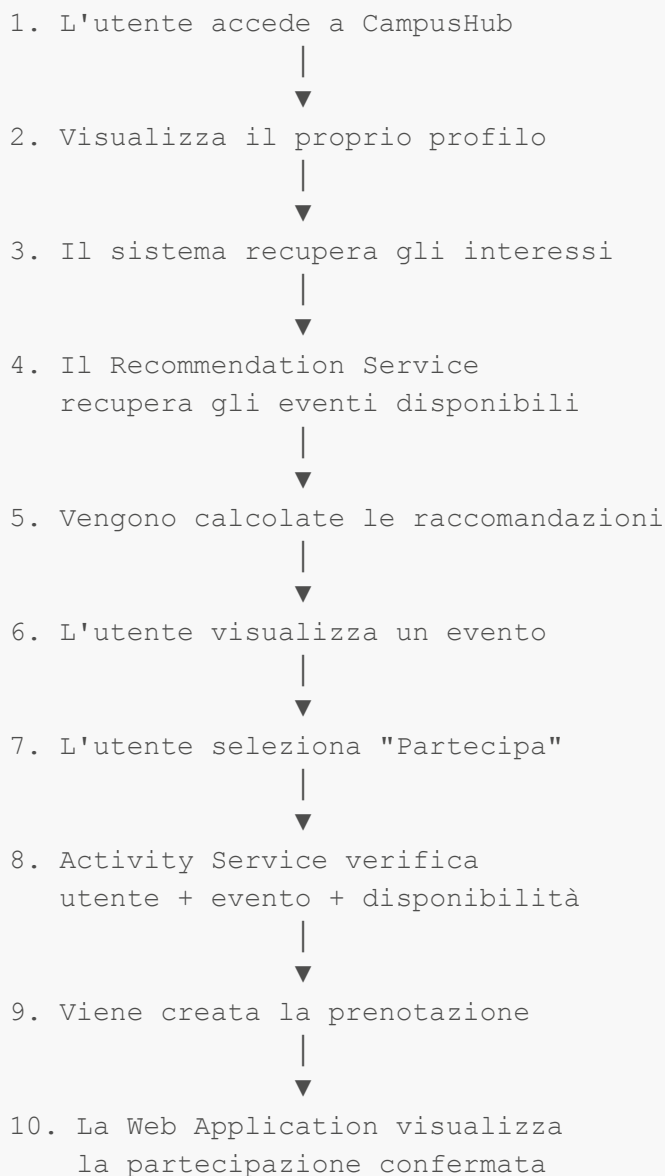
user.service.url=http://localhost:8082

event.service.url=http://localhost:8081
```

Le porte possono essere definite in fase di integrazione.

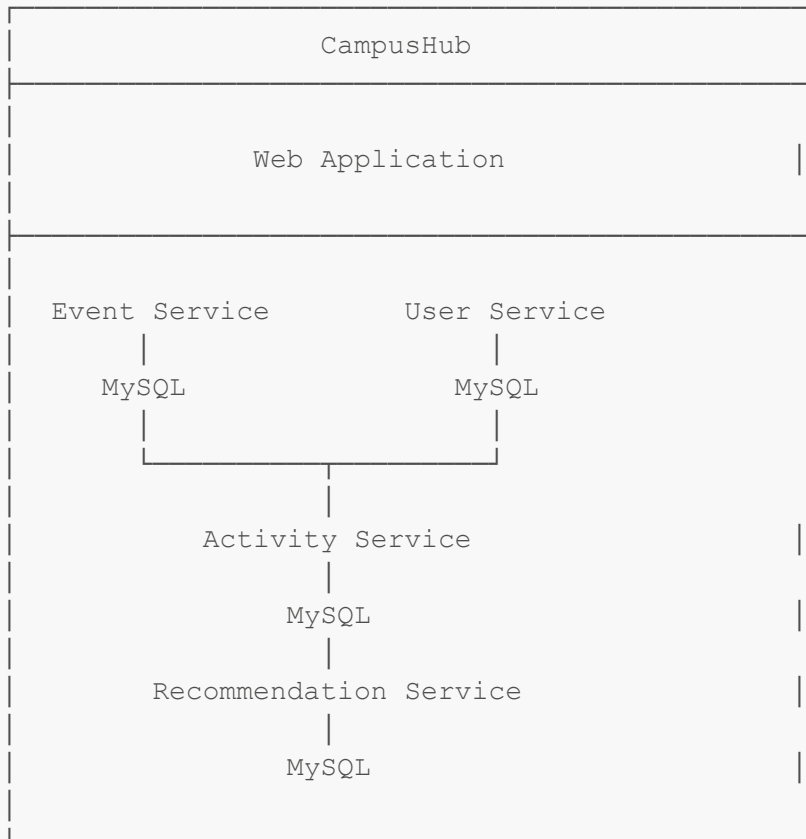
15. Flusso completo dell'applicazione

Il funzionamento complessivo previsto è:



16. Risultato finale

Il risultato deve essere una piattaforma web funzionante composta da:



La piattaforma deve consentire l'integrazione completa dei cinque componenti attraverso API REST, con dati persistenti su MySQL e interfaccia web accessibile tramite browser.